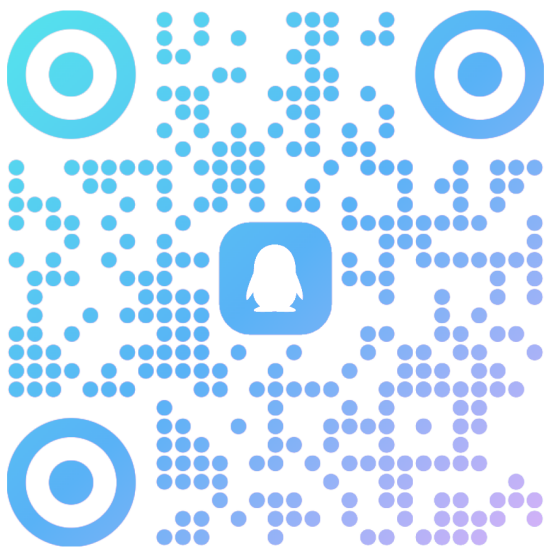




福州大学-2025编译...

群号: 1037351079



扫一扫二维码，加入群聊



编译原理

2025 春

任课教师: 柯嵩宇

Email: songyuke@fzu.edu.cn

2025-03-18

作业1完成情况

- 学生总数114
- 28人未交
- 问题汇总
 - 贴所有代码
 - 只有代码，没有分析
 - PDF没有嵌入字体，中文字符都没了

课前复习

四类文法

Grammar ⇄	Languages ⇄	Recognizing automaton ⇄	Production rules (constraints) ^[a] ⇄	Examples ^{[5][6]} ⇄
Type-3	Regular	Finite-state automaton	$A \rightarrow a$ $A \rightarrow aB$ (right regular) or $A \rightarrow a$ $A \rightarrow Ba$ (left regular)	$L = \{a^n n > 0\}$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \rightarrow \alpha$	$L = \{a^n b^n n > 0\}$
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$	$L = \{a^n b^n c^n n > 0\}$
Type-0	Recursively enumerable	Turing machine		$L = \{w w \text{ describes a halting Turing machine}\}$

绝大多数编程语言的语法都是上下文无关文法，但如果要考虑语义信息的话，它就变成了上下文相关语言了

正则语言

- 令 r 为一个正则表达式, 则 $L(r)$ 为其所定义的正则语言

[illegible]

词法分析器

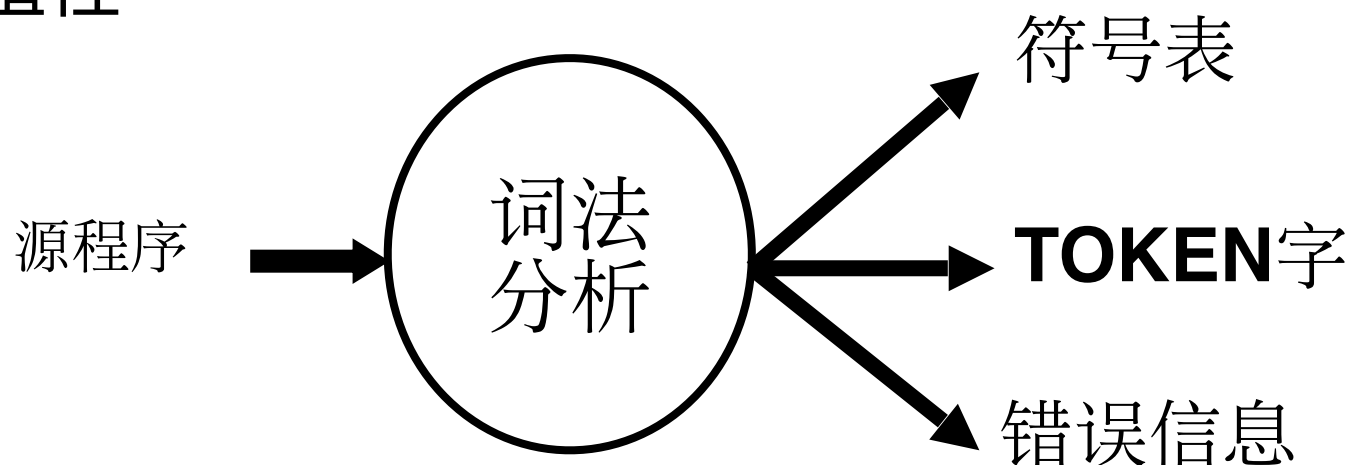
- 词法分析的任务
 - 逐行扫描源程序，识别出各个单词，确定单词的类型。同时将识别出的单词转换成统一的机内表示——词法单元(简称词元，英文：token)形式。
- 词法分析的主要流程
 - 去除注释、空白字符
 - 建立符号表
 - 识别词素，抽象成记号
 - 处理字面值
 - 发现词法错误，建立错误消息与源程序的位置联系
 - 将记号提交给语法分析器

词法分析器的输出

- 词法分析器的输出：
 - <单词种别, 单词自身的值>
- 程序语言单词的种类：
 - 关键字、标识符、常数、运算符、分界符
- 值
 - 标识符id、指针
 - 常数值

独立词法分析阶段的优势

- 结构简洁、清晰和条理化，有利于简化编译器的设计
- 提高效率
- 增强可移植性



词法分析器的实现方法

- 使用程序设计语言手写
 - C、C++、Java、
 - 汇编
- 使用词法分析器生成器，从基于正则表达式的词法描述自动产生

手工实现

- 基于待识别字符串前缀区分识别模式
 - 数字开头->常数字面值
 - 单引号/双引号——字符串or字符常量识别模式
 - 三引号——跨行字符串识别模式
 - “/”符号——除法还是注释？
 - A-Za-z_作为首个字符——标识符或关键字
 - 运算符
 - 大嘴法
 - 语法分析进一步处理

一站式词法语法分析器生成工具ANTLR

- ANTLR(**A**nother **T**ool for **L**anguage **R**ecognition)
 - 一个强大的解析器生成器，用于读取、处理、执行或翻译结构化文本或二进制文件。
 - 它被广泛用于构建语言、工具和框架。
 - ANTLR 根据语法定义生成解析器，解析器可以构建和遍历解析树。
 - 所有编程语言的语法，都可以用ANTLR来定义。
 - ANTLR提供了大量的官方 grammar 示例，包含了各种常见语言，比如 Java、SQL、Javascript、PHP等等。

Ref: <https://juejin.cn/post/7018521754125467661>

语法分析器

理论基础——上下文无关文法

- 上下文无关文法 (**C**ontext-**F**ree **G**rammar, **CFG**)

如何表示文法？

- 数学定义

- $G = (V_N, V_T, S, P)$
- V_N 是非终结符号集合
- V_T 是终结符号集合，可以理解为最终字符，其中有一个特殊符号 ε ，用于表示空
- S 是初始符号
- P 是产生式的集合

- 产生式/推导式

$$A \rightarrow B \mid C \mid D \mid E \mid \dots$$

文法的分类（乔姆斯基分类法）

- 2类文法——上下文无关文法（Context-free Grammar, CFG）

- 终结符号只能出现在生成式的右边

$$S \rightarrow AB$$

$$A \rightarrow HE$$

$$H \rightarrow \delta$$

$$E \rightarrow \tau$$

$$B \rightarrow HSE$$

一个句子的语法

He gave me a book.

<句子> → <主语> <谓语> <间接宾语> <直接宾语>

<主语> → <代词>

<谓语> → <动词>

<间接宾语> → <代词>

<直接宾语> → <冠词> <名词>

<代词> → He

<代词> → me

<名词> → book

<冠词> → a

<动词> → gave

用语法生成句子的过程

<句子>

⇒<主语><谓语><间接宾语><直接宾语>

⇒<代词><谓语><间接宾语><直接宾语>

⇒He <谓语><间接宾语><直接宾语>

⇒He <动词> <间接宾语><直接宾语>

⇒He gave <间接宾语><直接宾语>

⇒He gave <代词> <直接宾语>

⇒He gave me <直接宾语>

⇒He gave me <冠词><名词>

⇒He gave me a <名词>

⇒He gave me a book

文法的分类（乔姆斯基分类法）

- 2类文法——上下文无关文法（Context-free Grammar, CFG）

- 终结符号只能出现在生成式的右边

$$S \rightarrow AB$$

$$A \rightarrow HE$$

$$H \rightarrow \delta$$

$$E \rightarrow \tau$$

$$B \rightarrow HSE$$

- 尖括号表示语法成分，即非终结符
- 英语单词是词法单元，即终结符

如何表示文法？

- 数学定义

- $G = (V_N, V_T, S, P)$
- V_N 是非终结符号集合
- V_T 是终结符号集合，可以理解为最终字符，其中有一个特殊符号 ε ，用于表示空
- S 是初始符号
- P 是产生式的集合

- 产生式/推导式

$$A \rightarrow B \mid C \mid D \mid E \mid \dots$$

编程语言的语法

- 数学定义

- $G = (V_N, V_T, S, P)$
- V_N 是**语法成分**，也称语法变量，必定在某个生成式左边出现
- V_T 是终结符号集合，**词法单元 (token)**，为源代码中具体的一个字符串，可以为空，只能在生成式**右边**出现
- S 是初始符号，最高层次的语法成分，对于编程语言来说，可以是**program**或者**module**
 - 通常是第1条产生式规则左部的文法符号
- P 是产生式的集合

- 产生式/推导式

$$A \rightarrow B|C|D|E|\dots$$

例题

- 【例2-5】 定义只含+,*的算术表达式的文法

$$G = (\{id, +, *, (,)\}, \{E\}, P, E)$$

$$G(E): E \rightarrow E + E$$

$$P = \{$$

$$E \rightarrow E + E,$$

$$E \rightarrow E * E$$

$$E \rightarrow E * E,$$

$$E \rightarrow (E)$$

$$E \rightarrow (E),$$

约定：不引起歧义的前提下可以只写产生式

$$E \rightarrow id$$

$$G(E): E \rightarrow E + E \mid E * E \mid (E) \mid id$$

}

相同左部可简写，称之为候选式

例题

- 已知文法 $G(S): S \Rightarrow (S) \mid a$
- 请说明文法终结符集， 非终结符集， 开始符各是什么？
- 请说明本文法描述了什么？

例题

$list \rightarrow list + digit$

$list \rightarrow list - digit$

$list \rightarrow digit$

$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

前三条规则可以简写成什么？

最后一条规则相当于多少条规则？

语法的推导(Derivation)

- 给定文法 $G = (V_T, V_N, P, S)$, 若 $A \rightarrow \gamma \in P$ 且 $\alpha, \beta \in (V_T \cup V_N)^*$, 则文法符号串 $\alpha A \beta$ 可重写为 $\alpha \gamma \beta$, 记作

$$\alpha A \beta \Rightarrow \alpha \gamma \beta,$$

称 $\alpha A \beta$ 直接推导 $\alpha \gamma \beta$ 。

- 如果 $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$, 则称这个序列是从 α_1 到 α_n 的一个推导。
- 若存在一个从 α_1 到 α_n 的推导, 则称 α_1 可以推导出 α_n 。

例题

- 文法 $G(E): E \rightarrow E + E \mid E * E \mid (E) \mid id$ 的推导

$$E \Rightarrow (E) \Rightarrow (E + E) \Rightarrow (id + E) \Rightarrow (id + id)$$

推导的步数

- 推导长度：执行一步推导的次数
 - 一步推导
 - 每次只替换一个非终结符，用 $\Rightarrow$ 表示
 - 1步或若干步的推导
 - 用 $\Rightarrow^+$ 表示
 - 0步或若干步的推导
 - 用 $\Rightarrow^*$ 表示

例题

单步推导

给定文法中，如下哪个是合法的推导

$$S \rightarrow aXa$$

$$X \rightarrow \varepsilon \mid bY$$

$$Y \rightarrow \varepsilon \mid cXc \mid d$$

☐ S
aXa
abYa
acXca
acca

☐ S
aa

不是单步推导

☒ S
aXa
abYa
abcXca
abcbYca
abcbdca

☐ S
aXa
abYa
abcXcda
abccda

文法、语言、句型、句子

- 文法定义的语言

- 从开始符号出发，利用推导能得到的所有终结符号串的集合

- 句型

- 如果 $S \Rightarrow^* \alpha, \alpha \in (V_T \cup V_N)^*$, 则 α 为文法 G 的句型

- 句子

- 如果 $S \Rightarrow^+ \alpha, \alpha \in V_T^*$, 则 α 为文法 G 的句子

S 是文法的起始符号

- 语言

- 文法 G 的所有句子集合成为**文法 G 生成的语言**，记为 $L(G)$

$$L(G) = \{\alpha \mid S \Rightarrow^+ \alpha, \alpha \in V_T^*\}$$

- 上下文无关文法生成的语言称为**上下文无关语言**

例题

- 请证明 $(i * i + i)$ 是文法 $G(E): E \rightarrow i \mid E + E \mid E * E \mid (E)$ 的句子

写一个从起始符号到句子的推导式

文法和语言

- 文法和语言的关系
 - 给定一个文法，就从结构上唯一地确定其语言
 - 给定一种语言，能确定其文法，但文法不唯一
- 等价文法
 - 设 G_1 和 G_2 是两个文法，若 $L(G_1) = L(G_2)$ ，称 G_1 与 G_2 为等价文法，可记作 $G_1 \equiv G_2$

例题

- 文法 $G(A)$: $A \rightarrow c \mid Ab$ 产生的语言是什么?

- 文法 $G(S)$:

$$S \rightarrow AB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

$G(S)$ 产生的语言是什么?

- 请给出产生语言为 $\{a^n b^n \mid n \geq 1\}$ 的文法

例题

- 请给出语言 $L = \{ab^n a \mid n \geq 1\}$ 对应的文法，至少3种
- 请给出语言 $L = \{a2nb \mid n \geq 1\}$ 对应的文法，至少2种

语法分析

$list \rightarrow list + digit$

$list \rightarrow list - digit$

$list \rightarrow digit$

$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

- 语法分析的任务
 - 输入终结符号串，找出从开始符号推导出该串的方法。若不存在合法的推导则意味存在语法错误。
- 例
 - 用右上角的文法推导出 $9-5+2$

语法分析树

WHY?

- 从一个句型到一个句型的推导往往不唯一，可以以图形方式描述一个句子的推导过程，称为**语法分析树**（**语法树, Syntax Tree**）
- 某一文法的语法树具有以下性质：
 - **根节点**是**开始符号**
 - **叶节点**是**终结符**（即 token）或 ϵ
 - **内部节点**（非叶结点）是**非终结符**
 - 如果应用了规则 $A \rightarrow x_1x_2\dots x_n$ ，则
 - A 是内部结点，该子树的根节点
 - $x_1, x_2, \dots, x_n$ 是其从左到右的子结点（内部节点或叶节点）
 - 全部叶子节点从左到右构成**开始符号**推导得到的**符号串**

例子

$list \rightarrow list + digit$

$list \rightarrow list - digit$

$list \rightarrow digit$

$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$list \Rightarrow list + digit$

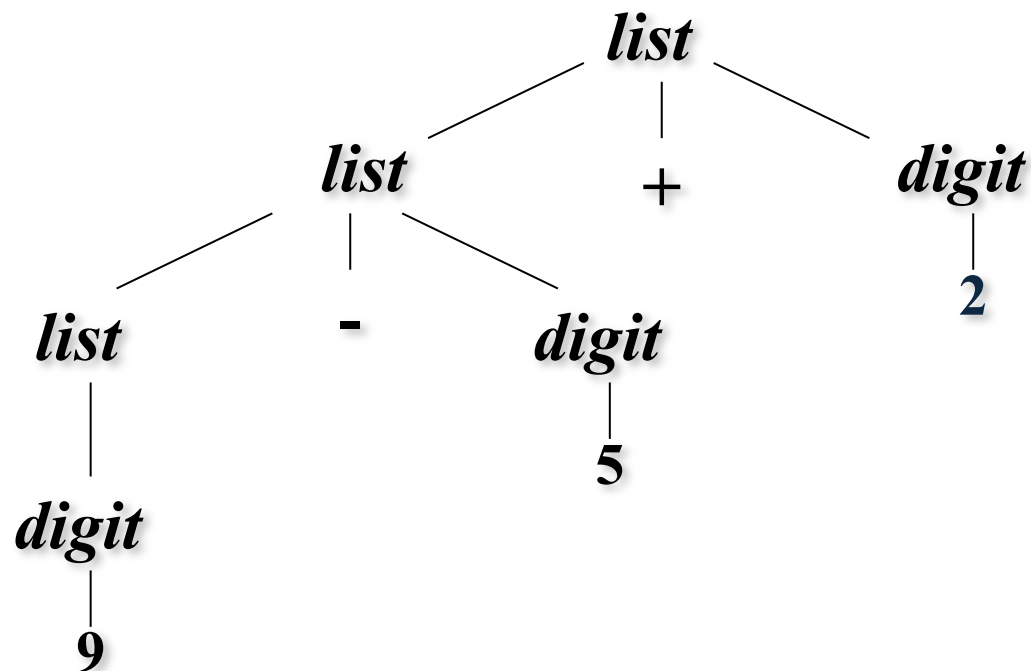
$\Rightarrow list - digit + digit$

$\Rightarrow digit - digit + digit$

$\Rightarrow 9 - digit + digit$

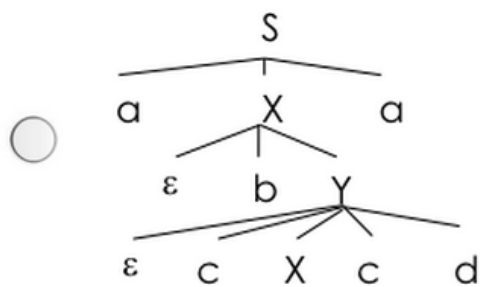
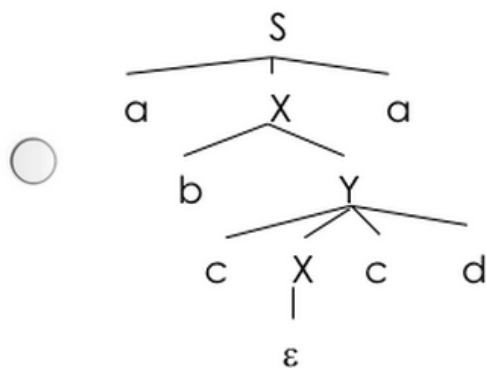
$\Rightarrow 9 - 5 + digit$

$\Rightarrow 9 - 5 + 2$



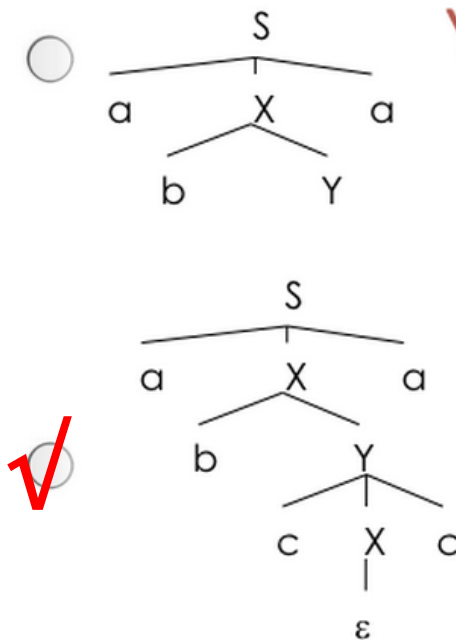
例题

【练2-7】 给定文法中，如下哪个是合法的语法树



$X \rightarrow \varepsilon \mid bY$

$Y \rightarrow \varepsilon \mid cXc \mid d$



例题

- 给定文法 $G(S): S \rightarrow SS + \mid SS * \mid a$
 - 请用它推导出 $aa + a *$
 - 画出语法分析树
 - 该语法生成的语言是什么?

语法的二义性

- 文法的二义性
 - 若文法存在某个句子对应两棵不同的**语法树**，则称该文法是**二义的**（**Ambiguous**）

示例

- 文法 $G(E): E \rightarrow i \mid E + E \mid E * E \mid (E)$ 是二义性文法
- 如何证明?
 - 画出句子 $(i * i + i)$ 的语法树

练习

- 证明文法 $S \rightarrow aSbS \mid bSaS \mid \varepsilon$ 是二义的

实例

- C语言
 - `if (a!=null) if (b==5) {} else {}`
- **Else是谁的?**
- If-then-else嵌套的二义性
 - 用{}确定语句体
 - 强制else与最近的没有else的if组合

二义性

- 造成二义性的原因
 - 文法中没有体现出结合律和优先级
- 解决方案
 - 在文法中体现结合律和优先级

整数的四则运算

- 四则运算存在两个优先级
 - 引入两个非终结符expr和term来对应不同的运算优先级层次
- 四则运算均为左结合
 - 抽象的非终结符应位于左边

expr $\rightarrow$ **expr** + **term** | **expr** - **term** | **term**

term $\rightarrow$ **term** * **factor** | **term** / **factor** | **factor**

factor $\rightarrow$ **digit** | (**expr**)

digit $\rightarrow$ 0 | 1 | 2 | ... | 9

练习

- 请写出一个支持C语言赋值运算符（右结合）的文法
 - 字面值：整数
 - 变量名：单个英文字母
 - 运算符
 - 左结合：+, -, *, /
 - 右结合：=
 - 其他：(,)
- $expr \rightarrow variable = expr \mid expr + term \mid expr - term \mid term$
- $term \rightarrow term * factor \mid term / factor \mid factor$
- $factor \rightarrow digits \mid variable \mid (expr)$
- $variable \rightarrow a \mid b \mid c \mid \dots \mid z$
- $digits \rightarrow (0 \mid 1 \mid 2 \mid \dots \mid 9)^*$

规律

- 按照优先级从低到高
 - $expr_0 \rightarrow \dots | expr_1$
 - $expr_1 \rightarrow \dots | expr_2$
 - $expr_2 \rightarrow \dots | expr_3$
 - $\dots$
 - $expr_{n-1} \rightarrow \dots | expr_n$
 - $expr_n \rightarrow num | variable | (expr_0)$
- 左结合
 - $expr_i \rightarrow expr_i \oplus expr_{i+1} | \dots | expr_{i+1}$
- 右结合
 - $expr_i \rightarrow expr_{i+1} \oplus expr_i | \dots | expr_{i+1}$

程序设计语言的CFG约定

- 不含 $P \rightarrow P$ 形式的产生式
- 每个非终结符 P 必须有用处，即存在如下推导

$$S \Rightarrow^* \alpha P \beta$$

$$P \Rightarrow^* \gamma$$

其中， $\gamma \in V_T^*$

作业

- 《编译原理》 P31
 - 2.2.1
 - 2.2.2
 - 2.2.3
 - 2.2.4
 - 2.2.5
 - 2.2.6